

Blink

"Fundamentos de Retrieval Systems"



Daniel Romero

Blink "Fundamentos de Retrieval Systems"

Alberto Souza

2026-05-02

Contents

O que você vai aprender com este curso?	i
Information Retrieval é o motor invisível por trás de toda busca	1
Tokenização é o primeiro passo para a máquina enten- der o que você escreveu	3
Preprocessing transforma texto ruidoso em dados pron- tos para análise	5
Tokens para a IA não são palavras — são identificadores numéricos sem significado semântico	7
Três modelos de retrieval, três formas de encontrar o que você procura	9
TF-IDF mede a importância real de cada palavra no seu conjunto de dados	11
Vetores transformam texto em matemática — e tornam a busca possível	13
Boolean Retrieval na prática: busca exata com índices invertidos	15
BM25 usa probabilidade para ranquear documentos por relevância	17
Retrieval é a base invisível do RAG e da IA generativa moderna	19
Resumo Final	21

Sobre este resumen 22

O que você vai aprender com este curso?

Você já parou para pensar no que acontece quando digita algo em um mecanismo de busca e, em segundos, recebe uma lista de resultados perfeitamente adaptados à sua pergunta? Essa “mágica” tem nome: **Information Retrieval**. E ela está por trás de praticamente tudo que envolve busca e recuperação de informação no mundo digital.

Neste curso, Daniel Romero guia você pelos fundamentos dos retrieval systems — os sistemas que tornam possível encontrar a agulha certa no palheiro de dados. Mais do que teoria, o curso entrega implementações práticas em Python que você pode adaptar e usar nos seus próprios projetos.

Ao longo destes blinks, você vai descobrir:

- Por que **tokenização** é o primeiro passo obrigatório para qualquer sistema que processa texto — e como implementá-la corretamente.
- Como o **pré-processamento** transforma texto bruto e ruidoso em dados prontos para análise.
- O que são **tokens** para os modelos de IA e por que eles enxergam o texto de forma completamente diferente de nós.
- Como funcionam os três grandes modelos de retrieval: **Boolean**, **Vector Space** e **Probabilistic**.
- Como o **TF-IDF** mede a importância real de cada palavra em relação a um conjunto de documentos.
- Como **vetores** e **cosine similarity** transformam a busca por texto em um problema matemático elegante.
- Como o **BM25** usa probabilidade para ranquear documentos por relevância.

- E por que tudo isso forma a base para construir sistemas de **RAG (Retrieval Augmented Generation)** com IA generativa.

Se você trabalha com dados, IA ou desenvolvimento de software e quer entender como sistemas de busca realmente funcionam por dentro, este resumo vai te dar uma base sólida e prática.

Information Retrieval é o motor invisível por trás de toda busca

Toda vez que você pesquisa algo no Google, busca um produto em um e-commerce ou pede algo a um assistente de voz, existe um sistema de Information Retrieval trabalhando nos bastidores. Em essência, Information Retrieval é sobre encontrar informações relevantes dentro de grandes volumes de dados com base em uma query do usuário.

Para entender como esses sistemas funcionam, pense em uma biblioteca enorme. Você precisa de um livro específico, mas existem milhões de volumes. Sem um sistema organizado, você levaria uma eternidade. Information Retrieval é justamente o sistema que permite localizar esse livro de forma rápida e eficiente.

Todo sistema de Information Retrieval se apoia em **três pilares fundamentais**:

Indexing é como montar um catálogo detalhado. Você pega todos os documentos, divide-os em partes menores — palavras, frases — e os armazena de uma forma que permita recuperação rápida. É o equivalente digital de um fichário de biblioteca onde cada palavra-chave aponta para os livros que a contêm.

Querying é o processo de consultar esse índice. Quando você digita uma busca, o sistema pega sua entrada, percorre o índice e retorna os documentos correspondentes. É como perguntar ao bibliotecário por livros sobre Machine Learning e ele consultar o catálogo para encontrá-los.

Ranking é o que diferencia um bom sistema de busca de um medíocre. Não basta encontrar documentos que correspondam à query — eles precisam ser ordenados por relevância, com os

mais importantes aparecendo primeiro.

“Não basta apenas encontrar os documentos que correspondem à query, eles precisam ser ordenados por relevância. Os algoritmos de ranking determinam quais documentos são mais relevantes e os colocam no topo dos resultados.”

Sempre que você projetar um sistema que envolva busca, pense nesses três pilares: como você indexa os dados, como processa as queries e como ranqueia os resultados. Essa estrutura se aplica desde um buscador simples em um site até pipelines complexos de RAG.

Tokenização é o primeiro passo para a máquina entender o que você escreveu

Antes que qualquer algoritmo possa analisar, indexar ou buscar em um texto, ele precisa quebrá-lo em pedaços menores. Esse processo se chama **tokenização**, e é literalmente o primeiro passo em qualquer pipeline de processamento de linguagem natural.

Tokenização é o processo de dividir um texto em partes menores — chamadas de **tokens** — que podem ser palavras, frases ou até caracteres individuais. Cada token representa uma unidade gerenciável do texto completo. Sem essa etapa, o algoritmo olharia para um bloco de texto como uma sequência indecifrável de caracteres.

Existem três formas principais de tokenizar:

Word Tokenization divide o texto em palavras individuais. É a forma mais comum e a base para a maioria das tarefas de NLP. A frase “Machine Learning é fascinante” vira os tokens: ["Machine", "Learning", "é", "fascinante"].

Sentence Tokenization divide o texto em sentenças completas. É útil quando você precisa analisar o texto no nível de frases — por exemplo, para resumos automáticos ou análise de sentimento por sentença.

Character Tokenization divide o texto em caracteres individuais. É menos comum, mas tem usos específicos em tarefas como correção ortográfica ou modelos que operam no nível de caracteres.

“Tokenization pode não parecer um processo simples, mas é um passo absolutamente crítico para lidar com dados de texto e analisá-los.”

Na prática, em Python, você pode usar o NLTK para implementar tokenização de forma direta. A função `word_tokenize` do NLTK é mais sofisticada que simplesmente dividir por espaços — ela lida adequadamente com pontuações, contrações e outros elementos linguísticos complexos.

Ao escolher sua estratégia de tokenização, considere o contexto. Tarefas diferentes exigem abordagens diferentes. Para retrieval e indexação, word tokenization geralmente é o ponto de partida. Teste, itere e ajuste conforme a necessidade do seu caso de uso.

Preprocessing transforma texto ruidoso em dados prontos para análise

Tokenizar o texto é só o começo. Se você jogar os tokens brutos diretamente em um algoritmo de busca, vai ter resultados ruins. O motivo? Texto real é cheio de ruído — pontuações, variações de maiúsculas e minúsculas, palavras irrelevantes. É por isso que o **preprocessing** (pré-processamento) é uma etapa tão fundamental.

O pré-processamento é onde você pega o texto bruto e o transforma em uma forma limpa e padronizada para análise. Três operações formam a base desse processo:

Conversão para minúsculas garante que “Machine” e “machine” sejam tratados como a mesma entidade. Sem isso, o algoritmo os consideraria tokens completamente diferentes, diluindo a relevância de ambos.

Remoção de caracteres não alfanuméricos elimina pontuações, símbolos e caracteres especiais que introduzem ruído na análise. Vírgulas, pontos e traços geralmente não carregam informação útil para busca.

Remoção de stopwords descarta palavras de conexão como “de”, “em”, “a”, “o” — palavras tão frequentes que aparecem em praticamente todos os documentos e, portanto, não ajudam a diferenciar um documento do outro. Porém, cuidado: em alguns contextos (como busca booleana), palavras como “e”, “ou” e “não” têm significado lógico e devem ser preservadas.

“Justamente nessa etapa é onde a maioria dos RAGs falham. Na construção dos chunks.”

Essa observação do curso é crucial. O pré-processamento não é apenas uma formalidade — é o ponto onde decisões aparentemente pequenas (remover ou não stopwords? manter ou não acentos?) impactam diretamente a qualidade dos resultados de busca.

Crie uma função de pré-processamento reutilizável que encapsule as três operações (lowercase, tokenização, filtragem). Use-a de forma consistente tanto nos documentos do índice quanto nas queries do usuário — aplicar regras diferentes nos dois lados é uma fonte comum de bugs em sistemas de retrieval.

Tokens para a IA não são palavras — são identificadores numéricos sem significado semântico

Quando você escreve “computadores” em uma query, pode parecer uma palavra simples. Mas para um modelo de IA como o GPT, essa palavra pode virar dois ou mais tokens. E é aí que muita gente se confunde: os tokens que a IA vê não são as palavras que nós vemos.

O tokenizer da OpenAI ilustra isso de forma clara. Ao inserir a frase “Machine Learning é um campo da inteligência artificial que permite que computadores aprendam padrões a partir de dados”, o modelo não processa cada palavra como uma unidade. Palavras como “computadores” e “programados” são quebradas em múltiplos tokens. Palavras mais curtas e comuns tendem a ser um único token.

Cada token recebe um **Token ID** — um número identificador. O ponto final, por exemplo, sempre recebe o mesmo ID (13 no modelo GPT-4). Mas esses IDs **não têm significado semântico**. O fato de dois IDs serem numericamente próximos não significa absolutamente nada — eles são apenas identificadores, como números de matrícula.

Um detalhe importante: o sistema de tokenização evoluiu significativamente entre gerações de modelos. A mesma frase em português que gerava 90 tokens no GPT-3 gera apenas 50 tokens no GPT-4. As versões mais antigas nem reconheciam acentos corretamente, quebrando palavras acentuadas em múltiplos tokens. As versões mais recentes compactam muito mais.

“Esses IDs, token IDs, ainda não têm relação alguma com embeddings, com vetores.”

Essa distinção é fundamental: tokens e Token IDs são a primeira camada de processamento. Embeddings e vetores — que carregam significado semântico — são uma camada posterior e completamente diferente.

Não se preocupe em decorar como tokenizers funcionam internamente — os modelos fazem essa contabilidade automaticamente. O importante é entender que tokens não são palavras, que palavras em idiomas com acentos e caracteres especiais podem gerar mais tokens, e que isso afeta custo e limites de contexto nas APIs.

Três modelos de retrieval, três formas de encontrar o que você procura

Nem toda busca é igual. Às vezes você sabe exatamente o que quer — um livro específico, com critérios rígidos. Outras vezes, está explorando, aberto a descobertas. É por isso que existem diferentes modelos de retrieval, cada um otimizado para um tipo de necessidade.

O **Boolean Retrieval Model** funciona como um sistema de filtros rígidos. Você especifica critérios exatos usando operadores lógicos: AND, OR e NOT. Buscar “Smartphone AND Samsung AND menos de mil” retorna apenas produtos que atendem a **todos** os critérios simultaneamente. É tudo ou nada — ou o documento atende, ou não. Não existe meio-termo nem classificação por relevância. É simples, preciso e previsível.

O **Vector Space Model** é mais flexível. Em vez de filtrar por critérios binários, ele entende que uma busca como “smartphone com boa câmera para fotos” pode significar coisas diferentes para pessoas diferentes. Ele classifica os documentos com base em quão bem eles combinam com a descrição, usando uma medida matemática de similaridade. É ótimo para descobrir opções que você não havia considerado, mas que ainda são relevantes.

O **Probabilistic Retrieval Model** funciona como um sistema de recomendação inteligente. Ele analisa padrões — compras anteriores, comportamento de usuários similares — e calcula a **probabilidade** de um documento ser relevante para você. Quanto mais dados o sistema tem, melhor ele fica.

“Você usaria o Boolean Model se tivesse requisitos muito específicos e não negociáveis. O Vector Space Model se quisesse ver uma variedade de opções. E o Probabilistic Model se estivesse procurando inspiração com base no que outros usuários com perfil similar compraram e gostaram.”

Na prática, escolha o modelo com base na natureza da busca. Filtrados rígidos? Boolean. Busca por similaridade semântica? Vector Space. Recomendação baseada em padrões? Probabilistic. Sistemas reais frequentemente combinam mais de um modelo.

TF-IDF mede a importância real de cada palavra no seu conjunto de dados

Se você quer saber quais palavras realmente importam em um documento, contar quantas vezes elas aparecem não é suficiente. Palavras como “de”, “a” e “que” aparecem centenas de vezes, mas carregam zero informação útil. O **TF-IDF** resolve esse problema combinando duas medidas complementares.

TF (Term Frequency) mede com que frequência um termo aparece em um documento específico. A fórmula é simples: o número de vezes que o termo aparece, dividido pelo número total de termos no documento. Se “Machine” aparece 1 vez em um documento de 6 termos, o TF é $1/6$.

IDF (Inverse Document Frequency) mede a relevância do termo no corpus inteiro. Usa-se o logaritmo do número total de documentos dividido pelo número de documentos que contêm aquele termo. Palavras que aparecem em poucos documentos recebem IDF alto — são mais discriminantes. Palavras que aparecem em todos os documentos recebem IDF baixo ou zero.

A pontuação final é simplesmente **TF × IDF**. E o resultado é revelador.

Considere três documentos sobre Machine Learning. A palavra “Machine” aparece em dois dos três documentos. Seu TF-IDF para o documento 1 é 0.029 — um valor baixo, porque a palavra é relativamente comum no corpus. Já a palavra “fascinante” aparece em apenas um documento. Seu TF-IDF é 0.08 — quase três vezes maior. Se alguém buscar por “fascinante”, o sistema sabe exatamente qual documento retornar.

“Palavras comuns como ‘de’, ‘são’, ou nesse exemplo, ‘learning’, frequentemente recebem valores de TF-IDF muito baixos ou até zero. Essas palavras são chamadas de stopwords e geralmente são removidas do corpus antes do processamento.”

O TF-IDF é elegante na simplicidade: palavras frequentes demais no corpus perdem importância, palavras raras ganham destaque. Essa mesma lógica se conecta diretamente com RAG — quando você domina TF-IDF, entende por que certos chunks são recuperados e outros não.

Vetores transformam texto em matemática — e tornam a busca possível

A ideia mais poderosa por trás do Vector Space Model é simples: transformar texto em vetores numéricos que podem ser comparados matematicamente. Quando você entende isso, entende como funciona toda busca moderna.

Na implementação prática com o **TfidfVectorizer** do scikit-learn, o processo funciona assim: você pega seus documentos pré-processados, aplica `fit_transform` e recebe uma **matriz TF-IDF**. Em um exemplo com 11 documentos, essa matriz tem dimensões 11×106 — cada documento é representado como um vetor de 106 dimensões, onde cada dimensão corresponde a um termo único do corpus.

Quando uma query chega — digamos, “machine learning” — ela passa pelo mesmo processo. O vectorizer cria um vetor de 106 dimensões para essa query, mesmo que ela tenha apenas duas palavras. As dimensões que não correspondem a termos da query recebem valor zero.

Com os vetores de documentos e da query em mãos, entra a **Cosine Similarity**. Essa métrica mede o ângulo entre dois vetores — quanto menor o ângulo, mais similares eles são. Documentos cujo vetor aponta na mesma “direção” que o vetor da query recebem pontuação alta.

O código é surpreendentemente enxuto: `cosine_similarity(tfidf_matrix, query_vector)` retorna um array de scores. Aplique `flatten()` para converter a matriz 2D em um array simples, ordene de forma decrescente e você tem os documentos ranqueados por relevância.

“A ideia central aqui é transformar os dados brutos em um formato que possa ser manipulado matematicamente, o que nos permite medir similaridades entre documentos e queries.”

A lógica geral de qualquer retrieval é sempre a mesma: vetorize os documentos, vetorize a query, meça a distância. Essa mesma lógica se aplica tanto ao TF-IDF clássico quanto a embeddings modernos com modelos de linguagem — muda a sofisticação do vetor, mas o princípio permanece.

Boolean Retrieval na prática: busca exata com índices invertidos

Enquanto o Vector Space Model trabalha com graus de similaridade, o **Boolean Retrieval Model** opera na lógica binária: um documento ou corresponde à query ou não. Pode parecer limitado, mas para buscas com critérios não negociáveis, é exatamente o que você precisa.

Na implementação prática, o curso usa a biblioteca **Whoosh** para construir um índice invertido — uma estrutura de dados que mapeia cada termo aos documentos que o contêm. O processo tem três etapas claras.

Primeiro, você define um **schema**: cada documento recebe um ID (título) e um campo de conteúdo indexável. Depois, cria o índice em um diretório no disco e alimenta-o com os documentos pré-processados. Por fim, constrói queries booleanas usando um **QueryParser**.

A query “machine AND learn” é parseada em uma cláusula AND com dois termos. O sistema percorre o índice e retorna apenas documentos que contêm **ambos** os termos. Se a query fosse “machine NOT learn”, retornaria documentos com “machine” que **não** contêm “learn”.

Um detalhe importante no pré-processamento para busca booleana: ao remover stopwords, é preciso **preservar os operadores lógicos**. No caso de conteúdo em português, isso significa manter “e”, “ou” e “não” na lista de exceções, já que eles funcionam como AND, OR e NOT nas queries.

“Estamos trabalhando com correspondências exatas das palavras, não dá para captar o significado. E isso é algo que está faltando aqui no momento.”

Essa é a limitação fundamental do modelo booleano: ele encontra palavras, não significados. Uma busca por “machine learning” não retorna documentos sobre “aprendizado de máquina” — mesmo que falem exatamente sobre o mesmo assunto.

Use Boolean Retrieval quando precisar de filtros absolutos — por exemplo, um sistema que precisa retornar apenas documentos que contenham obrigatoriamente certos termos regulatórios ou jurídicos. Para busca por significado, combine com Vector Space ou embeddings.

BM25 usa probabilidade para ranquear documentos por relevância

O modelo probabilístico parte de uma premissa intuitiva: para cada query, existe um conjunto ideal de documentos relevantes, e cada documento tem uma certa probabilidade de pertencer a esse conjunto. O **BM25** (Best Matching 25) é a implementação mais popular desse modelo, e ele está por trás de motores de busca reais como o Elasticsearch.

O BM25 considera vários fatores para calcular a relevância. Ele usa o **IDF** (frequência inversa de documentos) — o mesmo conceito do TF-IDF — para ponderar a importância dos termos. Mas vai além: ele normaliza pela frequência do termo no documento e pelo comprimento do documento, evitando que documentos longos sejam artificialmente favorecidos.

Na implementação prática com a biblioteca `rank_bm25`, o processo é direto: você tokeniza os documentos, inicializa o modelo BM25 com esses tokens e chama `get_scores` passando a query tokenizada. O resultado é um array de scores — quanto maior o valor, mais relevante é o documento.

Um ponto crucial destacado no curso: ao contrário do modelo booleano, **não remova stopwords ao usar BM25**. O modelo foi projetado para lidar com elas naturalmente através do IDF. Remover stopwords pode até prejudicar os resultados, porque certas palavras aparentemente irrelevantes carregam significado contextual que o BM25 sabe ponderar.

“Não devemos remover as stopwords ao construir BM25. O BM25 foi projetado para lidar com stopwords, e ele também usa o IDF, a frequência inversa dos documentos.”

Para ordenar os resultados, basta usar `numpy.argsort` no array de scores e inverter a ordem para que os documentos mais relevantes apareçam primeiro.

O BM25 é uma escolha excelente como baseline para qualquer sistema de retrieval. Ele é rápido, não precisa de GPU, não depende de embeddings e produz resultados surpreendentemente bons. Muitos pipelines de RAG modernos usam BM25 como primeiro estágio de filtragem antes de aplicar reranking com modelos mais sofisticados.

Retrieval é a base invisível do RAG e da IA generativa moderna

Todo o percurso do curso — tokenização, pré-processamento, TF-IDF, Boolean Retrieval, BM25 — converge para um ponto: esses são os fundamentos sobre os quais se constrói o **RAG (Retrieval Augmented Generation)**, uma das técnicas mais importantes da IA generativa atual.

O RAG funciona de forma simples em conceito: em vez de depender apenas do conhecimento interno de um modelo de linguagem (que pode estar desatualizado ou ser impreciso), você **recupera** documentos relevantes de uma base de conhecimento e os passa como contexto para o modelo gerar uma resposta. A qualidade dessa resposta depende diretamente da qualidade do retrieval.

Quando o retrieval falha, o modelo recebe contexto irrelevante e gera respostas ruins — é o famoso “garbage in, garbage out”. E como o curso destaca, a maioria dos RAGs falha justamente na etapa de pré-processamento e construção dos chunks. Não no modelo de linguagem.

Cada técnica vista no curso tem seu papel nesse ecossistema:

- **Tokenização e preprocessing** definem como os dados são preparados e indexados — e erros aqui se propagam para todas as etapas seguintes.
- **TF-IDF e Vector Space Model** são a base conceitual para entender como embeddings modernos funcionam — a lógica de vetorizar texto e medir similaridade é a mesma.
- **Boolean Retrieval** serve como filtro rígido quando você precisa de critérios não negociáveis.

- **BM25** continua sendo usado como componente de retrieval em sistemas de produção, frequentemente combinado com embeddings em abordagens híbridas.

“Quando formos construir uma aplicação real, o que importa é pegar o conteúdo e o contexto e então passar isso para o LLM para gerar uma resposta satisfatória.”

Antes de investir em embeddings sofisticados e modelos caros, domine os fundamentos. Um BM25 bem configurado com bom pré-processamento pode superar um sistema de embeddings mal implementado. Construa sobre bases sólidas.

Resumo Final

Os fundamentos de retrieval systems formam a base para qualquer trabalho sério com busca, processamento de linguagem natural e IA generativa. Aqui está o que você aprendeu:

Mensagem Central: Antes de um LLM gerar qualquer resposta, os dados certos precisam ser encontrados e entregues a ele. Retrieval é essa camada fundamental — e dominá-la é o que separa sistemas de IA que funcionam de sistemas que alucinam.

O pipeline começa com **tokenização** (quebrar texto em unidades processáveis), passa pelo **pré-processamento** (normalizar e limpar os dados), e chega à **indexação e busca** usando modelos como Boolean Retrieval (correspondência exata com lógica AND/OR/NOT), TF-IDF e Vector Space Model (similaridade vetorial com cosine similarity), ou BM25 (ranking probabilístico por relevância).

Cada modelo tem seu lugar: Boolean para critérios absolutos, TF-IDF para medir importância relativa de termos, BM25 para ranking probabilístico robusto. Sistemas reais frequentemente combinam mais de uma abordagem.

Conselho prático

Comece pelos fundamentos. Um pré-processamento bem feito com BM25 pode produzir resultados melhores que embeddings sofisticados mal configurados. Entenda tokenização, domine TF-IDF, implemente BM25 — e só então avance para embeddings e modelos neurais. A base sólida vai te poupar muito tempo e frustração.

Sobre este resumo

Este resumo foi gerado a partir do curso **“Fundamentos de Retrieval Systems”** de **Daniel Romero**.

O curso cobre os conceitos fundamentais de Information Retrieval — tokenização, pré-processamento, modelos de busca (Boolean, Vector Space, Probabilistic) e suas implementações práticas em Python — fornecendo a base necessária para compreender e construir sistemas de RAG (Retrieval Augmented Generation).

Este blink foi criado para oferecer uma visão rápida e prática dos principais conceitos do curso. Para aprofundamento com exercícios e código, recomendamos o curso completo.